

Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země



Jan Bartůšek, Petr Havel

Aplikace počítačové kartografie

Geometrické vyhledávání bodu

1. Zadání

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, \dots, P_n\}$, analyzovaný bod q .

Výstup: P_i , $q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q .

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

Hodnocení:

Krok	Hodnocení
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm.	+10b
Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu.	+5b
Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu.	+5b
Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.	+2b
Zvýraznění všech polygonů pro oba výše uvedené singulární případy.	+3b
Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).	+10b
Řešení pro polygony s dírami.	+10b
Načtení vstupních dat ze *.shp.	+10b
Max celkem:	65b

2. Bonusové úlohy a bodové hodnocení

V rámci řešení se podařilo úspěšně implementovat následující bonusové úlohy:

Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu (10b)

Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm (+10b)

Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu (+5b)

Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu (+5b)

Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem (+2b)

Zvýraznění všech polygonů pro oba výše uvedené singulární případy (+3b)

Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu) (+10b)

Řešení pro polygony s dírami (+10b)

Načtení vstupních dat ze shp (+10b)

Celkové možné hodnocení: 65 bodů

3. Popis a rozbor problému + vzorce

Při vyhledávání indikujícího polygonu pro bod q aplikace pracuje ve dvou fázích: globální proceduře (rychlé odfiltrování kandidátů) a lokální proceduře (exaktní geometrický test).

Špagetový datový model

Data jsou reprezentována čistě geometricky jako seznamy souřadnic bez explicitně vyjádřených topologických vztahů (sousedství hran, sdílení uzlů). Pro podporu děr je model strukturován jako vícerozměrné pole, kde každý polygon drží pole prstenců (první prstenec je vnější ohraničení, ostatní jsou díry).

Globální vyhledávání: Min-Max Box (Bounding Box)

Před spuštěním lokálních testů se spočítají extrémní souřadnice vnějšího prstence mnohoúhelníku P :

$$X_{min} = \min_j \{x_j\}, \quad X_{max} = \max_j \{x_j\}, \quad Y_{min} = \min_j \{y_j\}, \quad Y_{max} = \max_j \{y_j\}$$

Mnohoúhelník je okamžitě odmítnut jako neincidentní, pokud bod $q = [x_q, y_q]$ nesplňuje konjunkci podmínek:

$$(x_q \geq X_{min}) \wedge (x_q \leq X_{max}) \wedge (y_q \geq Y_{min}) \wedge (y_q \leq Y_{max})$$

Lokální procedura A: Ray Crossing Algorithm

Algoritmus je založen na Jordanově větě o křivce. Bodem q je veden horizontální paprsek směrem do kladného nekonečna ($y = y_q, x > x_q$). Spočítá se celkový počet průsečíků k tohoto paprsku s hranami polygonu.

Pro zjednodušení rovnic se provádí redukce souřadnic vrcholů vůči bodu q :

$$x'_i = x_i - x_q, \quad y'_i = y_i - y_q$$

Paprsek pak splývá s kladnou osou x' ($y' = 0, x' > 0$). Hrana mezi redukovánými vrcholy $p'_i p'_{i+1}$ protíná paprsek, pokud platí podmínka křížení polorovin:

$$(y'_{i+1} > 0 \wedge y'_i \leq 0) \vee (y'_i > 0 \wedge y'_{i+1} \leq 0)$$

X-ová souřadnice průsečíku x'_m se určí z lineární interpolace:

$$x'_m = \frac{x'_{i+1}y'_i - x'_i y'_{i+1}}{y'_{i+1} - y'_i}$$

Pokud vyjde $x'_m > 0$, průsečík leží vpravo od testovaného bodu a inkrementuje se čítač k . Pokud je výsledné k liché, bod leží uvnitř.

Lokální procedura B: Winding Number Algorithm

Metoda zjišťuje, kolikrát se orientovaný průvodič z bodu q otočí kolem své osy při obejití celého obvodu polygonu. Výsledný sumační úhel ω definuje Winding Number Ω :

$$\Omega = \frac{1}{2\pi} \sum_{i=0}^{n-1} \omega(p_i, q, p_{i+1})$$

Pro výpočet jednotlivých přírůstků úhlů mezi dvěma po sobě jdoucími redukovánými vrcholy využíváme křížový součin (odpovídající orientaci) a skalární součin vektorů:

$$\text{cross} = x'_i y'_{i+1} - x'_{i+1} y'_i, \quad \text{dot} = x'_i x'_{i+1} + y'_i y'_{i+1}$$

$$\omega_i = \arctan 2(\text{cross}, \text{dot})$$

Pokud platí $\Omega = 1$ (respektive suma úhlů se blíží 2π), bod se nachází uvnitř mnohoúhelníku.

4. Popisy algoritmů formálním jazykem

Formální zápis Ray Crossing s ošetřením singularit

Algoritmus: getPointPolygonPositionRC(q, complex_pol)

Vstup: Testovaný bod q, komplexní mnohoúhelník tvořený prstenci [Ring_0 (vnější), Ring_1 ... (díry)]

Výstup: Pozice bodu (1 = uvnitř, 0 = vně, -1 = hrana, -2 = vrchol)

1. k = 0, tolerance = 5.0
2. PRO KAŽDÝ prstenec ring V komplex_pol OPAKUJ:
3. n = délka(ring)
4. PRO i OD 0 DO n-1 OPAKUJ:
5. xi = ring[i].x - q.x, yi = ring[i].y - q.y
6. xi1 = ring[(i+1)%n].x - q.x, yi1 = ring[(i+1)%n].y - q.y
7. // Test singularity na vrchol
8. POKUD hypot(xi, yi) <= tolerance PAK NAVRAŤ -2
9. // Test singularity na hranu
10. dx = xi1 - xi, dy = yi1 - yi
11. edge_length = hypot(dx, dy)
12. POKUD edge_length > 0 PAK:
13. distance_to_line = abs(xi * yi1 - xi1 * yi) / edge_length
14. POKUD distance_to_line <= tolerance PAK:
15. dot_product = -(xi * dx + yi * dy)
16. POKUD 0 <= dot_product A dot_product <= edge_length^2 PAK NAVRAŤ -1
17. // Standardní paprskový test
18. POKUD (yi1 > 0 A yi <= 0) NEBO (yi > 0 A yi1 <= 0) PAK:
19. xm = (xi1 * yi - xi * yi1) / (yi1 - yi)
20. POKUD xm > 0 PAK:
21. k = k + 1
22. POKUD (k % 2 == 1) PAK NAVRAŤ 1 JINAK NAVRAŤ 0

Formální zápis Winding Number s ošetřením děr

Algoritmus: `getPointPolygonPositionWN(q, complex_pol)`

Vstup: Testovaný bod q , komplexní mnohoúhelník `complex_pol`

Výstup: Pozice bodu (1 = uvnitř, 0 = vně, -1 = hrana, -2 = vrchol)

1. POKUD `complex_pol` je prázdný NEBO vnější obrys `complex_pol[0]` je prázdný PAK NAVRÁT 0
2. `outer_status = _getWindingNumberForRing(q, complex_pol[0])`
3. POKUD `outer_status != 1` PAK NAVRÁT `outer_status` // Bod je vně, na hraně nebo na vrcholu vnějšího obrysu
4. PRO KAŽDÝ prstenec hole V `vnitřní_díry complex_pol[1:]` OPAKUJ:
5. POKUD hole je prázdný PAK POKRAČUJ
6. `hole_status = _getWindingNumberForRing(q, hole)`
7. POKUD `hole_status == -1` NEBO `hole_status == -2` PAK NAVRÁT `hole_status` // Hrana/vrchol díry je hranou celého polygonu
8. POKUD `hole_status == 1` PAK NAVRÁT 0 // Bod spadl do díry, je tedy technicky vně polygonu
9. NAVRÁT 1 // Bod je uvnitř vnějšího obrysu a mimo všechny vnořené díry

5. Problematické situace a jejich rozbor + ošetření v kódu

Při převodu teorie do kódu vznikají numerické a geometrické singularity, které bez ošetření vedou k nesprávným výsledkům nebo pádům aplikace (např. dělení nulou). Uživatelské rozhraní vyžaduje zavedení prostorové tolerance `tolerance = 5.0 pixelů` pro kompenzaci nepřesnosti kliknutí myší.

A) Bod je totožný s vrcholem (Singularita 1)

Pokud paprsek prochází přesně vrcholem, standardní Ray Crossing by mohl průsečík započítat dvakrát nebo vůbec (podle sklonu sousedních hran). V aplikaci situaci předcházíme přímým testem Euklidovské vzdálenosti bodu od vrcholu:

$$d = \sqrt{(x_i - x_q)^2 + (y_i - y_q)^2} \leq \text{tolerance}$$

Ošetření v kódu (`algorithms.py`):

```
# Check if point is on vertex
if math.hypot(xi, yi) <= tolerance:
    return -2
```

B) Bod leží na hraně polygonu (Singularita 2)

Pokud bod leží přesně na vodorovné hraně, rovnice pro výpočet průsečíku selže kvůli dělení nulou ($y_i - y_{i+1} = 0$). Stav detekujeme výpočtem kolmé vzdálenosti bodu od přímky tvořené hranou:

$$\text{distance} = \frac{|x'_i y'_{i+1} - x'_{i+1} y'_i|}{\sqrt{(x'_{i+1} - x'_i)^2 + (y'_{i+1} - y'_i)^2}}$$

Abychom vyloučili situaci, kdy bod leží na přímce, ale mimo úsečku hrany, ověřujeme polohu průmětu pomocí skalárního součinu, který musí padnout do intervalu $\langle 0, \text{délka hran}^2 \rangle$.

Ošetření v kódu (algorithms.py):

```
# Check if point is on edge
dx = x1 - xi
dy = y1 - yi
edge_length = math.hypot(dx, dy)

if edge_length > 0:
    distance_to_line = abs(xi * y1 - x1 * yi) / edge_length
    if distance_to_line <= tolerance:
        dot_product = -(xi * dx + yi * dy)
        if 0 <= dot_product <= edge_length**2:
            return -1
```

C) Topologie vnořených prstenců (Díry)

U komplexních polygonů se topologické vnitřní otvory chovají inverzně. Pokud bod leží uvnitř díry, nachází se z hlediska mapy vně polygonu. Winding Number toto ošetřuje postupným vyhodnocením: bod se zkontroluje vůči vnějšímu prstenci a následně vůči všem dírám. Pokud je uvnitř jakékoliv díry, výsledek se invertuje na 0 (vně).

6. Vstupní data, formát vstupních dat, popis

Aplikace akceptuje dva typy vstupních dat:

1. **Interaktivní vektorový vstup:** Uživatel zadává vrcholy ručně klikáním na plátno (třída Draw). Levé tlačítko přidává lomové body do aktuálního prstence, pravé tlačítko prstencem uzavře a inicializuje kreslení vnitřní díry, prostřední tlačítko (kolečko) uzavře celou komplexní entitu a připraví strukturu pro nový samostatný polygon.
2. **Souborový formát ESRI Shapefile (.shp):** Jde o binární formát uchovávající geometrická data. Aplikace načítá data pomocí knihovny pyshp. V parseru filtrujeme typ objektu `shapeType == 5` (Polygon). Vektorové body jsou organizovány v poli `shape.points` a indexy, kde končí vnější obrysy a začínají díry, jsou načítány z pole `shape.parts`.

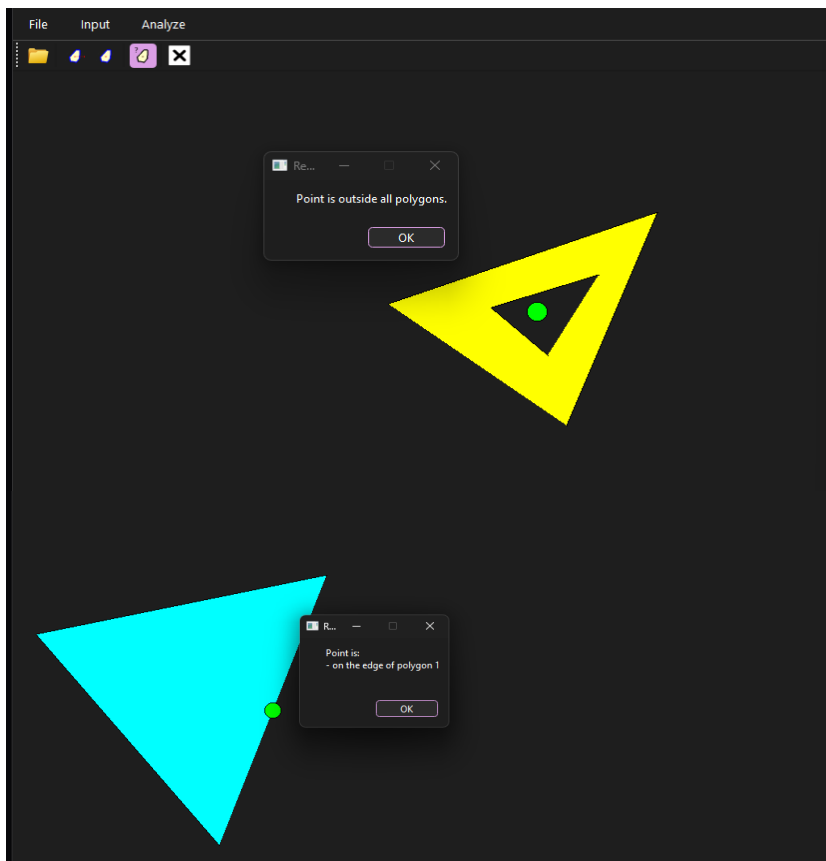
Pro vykreslení dat na obrazovku probíhá transformace geografických souřadnic na pixely plátna. Výpočtem globálního bounding boxu celé vrstvy se určí měřítko scale s 5% vnitřním okrajem a provede se převrácení osy (`y`), protože v počítačové grafice počátek leží v levém horním rohu, zatímco v kartografii roste osa směrem nahoru.

7. Výstupní data, formát výstupních dat, popis

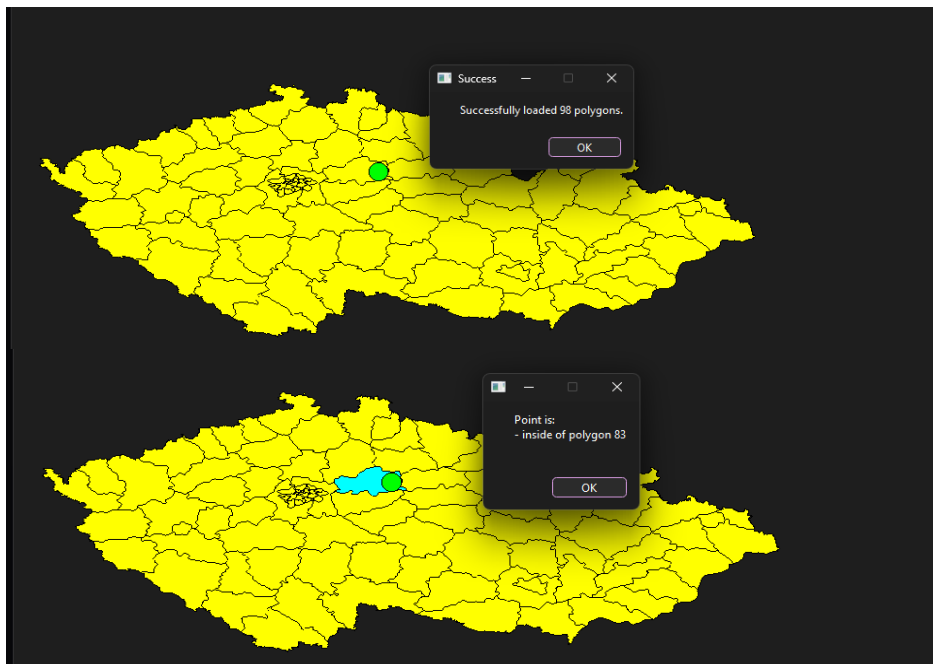
Výstupy aplikace jsou realizovány textovou a vizuální formou:

- **Vizuální interpretace:** Pokud je detekována incidence, index polygonu se zapíše do pole `__highlighted_indices` ve třídě `Draw`. Při překreslení (`paintEvent`) změní tyto vybrané polygony barvu výplně ze žluté na azurovou (`Qt.GlobalColor.cyan`). Testovaný bod q se vykresluje jako výrazný zelený kruh o poloměru 10 pixelů.
- **Textové hlášení (Formát GUI):** Výsledek analýzy zprostředkovává komponenta `QMessageBox`. Formát textu vypisuje seznam zasažených prvků a jejich přesný topologický stav (např. - inside of polygon 1 nebo - on the edge of polygon 3). Pokud bod neleží v žádném objektu, vypíše se hláška "Point is outside all polygons."

8. Printscreen vytvořené aplikace



Obrázek 1: Ručně nakreslené objekty



Obrázek 2: Načtený Shapefile soubor

9. Programátorská dokumentace a detailní rozbor kódu

Architektura aplikace striktně odděluje grafické uživatelské rozhraní, datový/vykreslovací model a výpočetní logiku. Program je rozdělen do tří nezávislých modulů: `algorithms.py`, `draw.py` a `MainForm.py`. Níže je rozebrána vnitřní mechanika klíčových tříd a metod.

9.1 Modul `draw.py` (Vykreslovací plátno a správa dat)

Třída `Draw` dědí z `QWidget` a slouží jako interaktivní plátno. Drží v sobě veškerá prostorová data (polygony i testovací bod) a stará se o jejich překreslování.

Datový model pro komplexní polygony: Aby bylo možné reprezentovat polygony s dírami, byl standardní datový typ Qt (`QPolygonF`) obalen do dvourozměrného pole (seznam seznamů).

```
class Draw(QWidget):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # List of polygons (index 0 is outer boundary, rest are holes)
        self.__polygons = [[QPolygonF()]]
        self.__q = QPointF(100, 100)
        self.__add_vertex = True
        self.__highlighted_indices = []
```

`__polygons`: Hlavní paměťová struktura. Nultý index (např. `self.__polygons[0][0]`) představuje vždy vnější hranici prvního polygonu. Pokud má polygon díry, ukládají se na indexy `[0][1]`, `[0][2]` atd.

`__add_vertex`: Logický přepínač. Pokud je `True`, uživatel myší kreslí mapu. Pokud je `False`, uživatel klikáním přesouvá testovací bod `__q`.

Zachytávání událostí myši (mousePressEvent): Tato metoda řídí logiku vkládání bodů. Zatímco levé tlačítko pouze přidává vrcholy do aktivního prstence, pravé a prostřední tlačítko se starají o topologii (vytváření děr vs. vytváření nových polygonů).

```
if self.__add_vertex:
    # Left click: add vertex
    if e.button() == Qt.MouseButton.LeftButton:
        self.__polygons[-1][-1].append(QPointF(x, y))

    # Right click: finish current ring, start drawing a hole
    elif e.button() == Qt.MouseButton.RightButton:
        if not self.__polygons[-1][-1].isEmpty():
            self.__polygons[-1].append(QPolygonF())

    # Middle click: finish whole polygon, start a completely new one
    elif e.button() == Qt.MouseButton.MiddleButton:
        if self.__polygons[-1][-1].isEmpty():
            self.__polygons[-1].pop()
        if self.__polygons[-1]:
            self.__polygons.append([QPolygonF()])
```

Vykreslování děr (paintEvent): Aby byly díry v polygonu skutečně průhledné a neschovávaly se pod barvou vnějšího obrysu, je použita třída QPainterPath. Celý komplexní polygon (vnější hranice i všechny díry) se vloží do jedné cesty, na kterou se aplikuje pravidlo OddEvenFill.

```
# Setup path for drawing holes
path = QPainterPath()
path.setFillRule(Qt.FillRule.OddEvenFill)

# Add all rings to path
for ring in complex_pol:
    if not ring.isEmpty():
        path.addPolygon(ring)

qp.drawPath(path)
```

9.2 Modul algorithms.py (Geometrické výpočty)

Tato třída neobsahuje žádné UI prvky, přijímá čistá geometrická data a vrací stavové kódy (1 = uvnitř, 0 = vně, -1 = hrana, -2 = vrchol).

Filtrace Min-Max Box (isInBoundingBox): Před spuštěním analytických metod se zjišťují extrémy x a y vnější hranice (díry pro obálku nejsou potřeba, proto se bere pouze `complex_pol[0]`). Metoda okamžitě vrací False, pokud bod leží mimo tento obdélník.

```
def isInBoundingBox(self, q: QPointF, complex_pol: list[QPolygonF]):
    # Quick filter using min-max box
    if not complex_pol or complex_pol[0].isEmpty():
        return False

    outer_ring = complex_pol[0]
    x_coords = [p.x() for p in outer_ring]
    y_coords = [p.y() for p in outer_ring]

    min_x, max_x = min(x_coords), max(x_coords)
    min_y, max_y = min(y_coords), max(y_coords)

    # Point is completely outside the box
    if q.x() < min_x or q.x() > max_x or q.y() < min_y or q.y() > max_y:
        return False
    return True
```

Ray Crossing s detekcí singularit: V rámci metody se cyklicky prochází všechny prstence (včetně děr, paprsková metoda zvládá díry nativně). Nejprve se ověřují singularity pomocí redukovaných souřadnic (x_i , y_i) vůči bodu q . Zde je ukázka detekce hrany pomocí kolmé vzdálenosti a skalárního součinu:

```
# Check if point is on edge
dx = x1 - xi
dy = y1 - yi
edge_length = math.hypot(dx, dy)

if edge_length > 0:
    distance_to_line = abs(xi * y1 - x1 * yi) / edge_length
    if distance_to_line <= tolerance:
        dot_product = -(xi * dx + yi * dy)
        if 0 <= dot_product <= edge_length**2:
            return -1
```

Samotný test průsečíku hrany a paprsku pak probíhá na ose X ($y = 0$). Kontroluje se křížení na ose Y a průsečík x_m musí ležet vpravo (> 0):

```
# Ray intersection logic (y = 0)
if (yi1 > 0 and yi <= 0) or (yi > 0 and yi1 <= 0):
    xm = (xi1 * yi - xi * yi1) / (yi1 - yi)
    if xm > 0:
        k += 1
```

Topologická kontrola pro Winding Number (getPointPolygonPositionWN): Protože Winding Number sčítá úhly, díry by mohly výsledek znehodnotit. Proto je algoritmus řízen touto nadřazenou metodou, která striktně vyžaduje, aby byl bod *uvnitř* vnějšího obrysu ($outer_status == 1$) a zároveň aby nebyl *uvnitř* žádné díry ($hole_status == 1$ vrací 0 = venku).

```
# 1. Check if point is inside the main polygon
outer_status = self._getWindingNumberForRing(q, complex_pol[0])
if outer_status != 1:
    return outer_status

# 2. Check if point is inside any hole
for hole in complex_pol[1:]:
    if hole.isEmpty():
        continue

    hole_status = self._getWindingNumberForRing(q, hole)
    # Point is on the edge of a hole
    if hole_status == -1 or hole_status == -2:
        return hole_status
    # Point is inside a hole -> outside of the polygon
    if hole_status == 1:
        return 0
```

9.3 Modul MainForm.py (Řízení aplikace a integrace)

Třída `Ui_MainForm` inicializuje grafické rozhraní, navěšuje události (Signály a Sloty) na prvky vygenerované z Qt Designeru a funguje jako hlavní Controller (přenos dat mezi plátnem a algoritmy). Načítání prostorových dat (.shp) a transformace souřadnic: Metoda `openFileClick` otevírá dialog pro výběr .shp souboru. Pomocí knihovny `shapefile` rozebírá geometrie do špagetového modelu. Klíčový je přepočítání měřítka. Získá se `bbox` celého souboru, vypočítá se poměr pro obrazovku 800x800 a osa Y se převrátí.

```
min_x, min_y, max_x, max_y = sf.bbox
width = max_x - min_x
height = max_y - min_y

# Calculate scale to fit 800x800 window
scale_x = 800 / width
scale_y = 800 / height
scale = min(scale_x, scale_y) * 0.95

for shape in shapes:

    for px, py in ring_points:
        # Translate, scale and flip Y axis
        screen_x = (px - min_x) * scale + 20
        screen_y = 850 - ((py - min_y) * scale + 20)
        ring.append(QtCore.QPointF(screen_x, screen_y))
```

Hlavní smyčka analýzy (`runAnalysis`): Metoda načte souřadnice `q` a pole polygonů z instance `Canvas`. Pro každý polygon zavolá instanci `Algorithms`. Úspěšné nálezy (stavy 1, -1, -2) formátuje do pole stavových zpráv a pole indexů odesílá zpět plátnu pro grafické zvýraznění (`setHighlightedPolygons`).

```
for i, complex_pol in enumerate(polygons):
    if not complex_pol or complex_pol[0].isEmpty():
        continue

    # Skip if point is outside bounding box
    if not a.isInBoundingBox(q, complex_pol):
        continue

    if algorithm_name == "Ray Crossing":
        result = a.getPointPolygonPositionRC(q, complex_pol)
    else:
        result = a.getPointPolygonPositionWN(q, complex_pol)

    # Save results
    if result == 1:
        highlighted_indices.append(i)
        status_messages.append("inside")
    elif result == -1:
        highlighted_indices.append(i)
        status_messages.append("on the edge")
```

10. Závěr, možné či neřešené problémy, náměty na vylepšení

Aplikace je stabilní a korektně řeší Point Location Problem včetně pokročilé topologie (vnořené díry) a ošetření okrajových stavů s přesností na zvolenou pixelovou toleranci.

Limity stávajícího řešení (Neřešené problémy): Aplikace využívá standardní kartézský souřadnicový systém v rovině. Z tohoto důvodu vykazuje chyby při načtení globálních map světa, kde některé polygony kříží mezinárodní datovou hranici (180. poledník – např. východní výběžek Ruska nebo velké polygony Antarktidy). V těchto souborech se geometrie polygonu "roztrhne" mezi záporný levý okraj a kladný pravý okraj mapy, což zcela znehodnotí výpočet Min-Max Boxu i protnutí paprsků. Pro školní a lokální datasety (např. administrativní dělení ČR) je však model plně stabilní.

Náměty na vylepšení: I když Bounding Box (Min-Max filtr) výrazně šetří procesorový čas, při načítání obrovských geografických vrstev (miliony prvků) je lineární procházení seznamu obálek v cyklu $O(n)$ stále neefektivní. Výrazného zrychlení by se dalo dosáhnout implementací hierarchických prostorových indexů, jako je R-Tree nebo Quadtree. Vyhledání potenciálních polygonů by pak kleslo z lineární složitosti na složitost logaritmickou $O(\log n)$.

11. Seznam literatury

1. BAYER, Tomáš. *Point Location Problem. Konvexní a nekonvexní oblasti. Ray Algorithm. Winding Number Algorithm* (Přednáška adk3_new.pdf, Katedra aplikované geoinformatiky a kartografie, Přírodovědecká fakulta Univerzity Karlovy).